

R3.01 développement web

R3.01 développement web

Date de publication 2026

Table des matières

1. Présentation	1
1. Objectifs	1
2. Environnement de travail	1
2.1. Utilisation d'un conteneur léger pour héberger notre serveur	1
2.2. Un premier code java	2
2.2.1. Structure d'un projet JEE	2
2.2.2. Nos premier codes	2
2.2.3. La compilation et le déploiement	3
2. JEE (servlet, JSP, JPA)	6
1. Apprentissage de java	6
1.1. Différences entre java et Kotlin	6
1.2. Un premier code en java	6
1.2.1. Un tutoriel	6
1.2.2. La promotion : Classe et héritage	7
1.2.3. Mise en oeuvre du design pattern DAO	10
2. Les servlet	12
2.1. Cycle de vie	12
2.2. Recevoir des paramètres	12
3. JSP	12
4. JPA	12
3. Une première application MVC	13
4. Utilisation du framework MVC spring boot	14

Liste des illustrations

2.1. Classe Etudiant	7
2.2. EtatMental	8
2.3. EtudiantAvecEtatMental	9
2.4. Etudiant avec comptage	10

Chapitre 1. Présentation

1. Objectifs

Dans la ressource R1.02:Développement d'interfaces web, vous avez appris à réaliser des pages web statiques. Dans cette ressource nous allons produire des pages Web dynamiques via un serveur HTTP (Hypertext Transfer Protocol). Le langage choisi est Java avec Jakarta Enterprise Edition (JEE), un langage de script impératif orienté objet. Pour la persistance de nos données nous utiliserons le SGBD (Système de Gestion de Base de Données) H2.

Nous disposons dans cette ressource de 5 créneaux de 1h20 de CM et de 22 créneaux de TD. Voici une progression indicative :

Semaines	CM	TD	TD de SAE	Évaluation	Objectifs
S40	1h20	2h40			Apprentissage JAVA
S41	1h20	4h			Apprentissage JAVA + Servlet
S42	1h20	4h			Servlet
S43	1h20	4h			JSP
S45	1h20	4h			JPA
S46				2h40	JPA
S47		4h			MVC
S48		2h40	1h20		MVC
S49			4h		Spring Boot
S50			1h20	2h40	Spring Boot
S51			?		Spring Boot
S1			?		Soutenance SAE

2. Environnement de travail

Nous allons créer et exécuter notre première servlet. Une servlet est une classe Java qui permet de créer dynamiquement des données au sein d'un serveur HTTP. Il nous faut donc un serveur HTTP et un moteur de servlet (conteneur web) qui prend en charge et gère les servlets. Nous allons utiliser tomcat (<https://tomcat.apache.org/>) un moteur léger.

2.1. Utilisation d'un conteneur léger pour héberger notre serveur

Lors du S4 vous aurez des cours de virtualisation qui vous permettront de réaliser les scripts présentés ici. Nous allons utiliser podman, un logiciel libre compatible avec docker permettant de lancer des applications dans des conteneurs logiciels.

Étant derrière un proxy, pour ne pas créer de déni de service (DoS), il vous faut créer un compte sur <https://www.docker.com/>. Ce compte sera utilisé pour le téléchargement des images.

Vous disposez des scripts suivants :

`docker-compose.yml`

script qui permet la création du conteneur et la définition du point de montage, le `./tomcat/webapps` sera monté dans `/usr/local/tomcat/webapps` du conteneur.

create.sh

script qui vous demande de vous authentifier et qui lance en arrière plan les services définis dans docker-compose.yml

down.sh

script qui stop les services avant de vous déloguer

terminal.sh

script qui permet d'obtenir un terminal dans le conteneur

Mettre les droits d'exécution sur les scripts bash (**chmod u+x *.sh**) puis tester l'URL `http://localhost:8888/`, vous devriez obtenir un code 404, le serveur répond mais n'a pas de données à fournir.

2.2. Un premier code java

Notre premier programme est un projet JEE et doit respecter une structure :

2.2.1. Structure d'un projet JEE

```
src
  ### main
  #   ### java
  #   #   ### iut
  #   #   ### nantes
  #   #   ### demo
  #   #   ### HelloServlet.java
  #   ### resources
  #   ### webapp
  #       ### WEB-INF
  #       #   ### web.xml
  #       ### index.jsp
  ### test
  ### java
  ### resources
```

src/main/java

Contient tout le code source Java

src/main/ressources

Contient les fichiers de configuration

src/main/webapp/

est la racine du contenu web : HTML, CSS, JS, JSP (Jakarta Server Pages), ...

src/main/webapp/WEB-INF

est un dossier protégé contenant le descripteur de déploiement (web.xml) et les pages sécurisées.

src/test/java

est le dossier contenant le code source des tests unitaires et d'intégration

2.2.2. Nos premier codes

2.2.2.1. Une servlet

Le constituant de base est la servlet, une classe Java qui permet de créer dynamiquement des données au sein d'un serveur HTTP. Les servlet disposent d'un cycle de vie et répondent à des requêtes HTTP.

Dans l'exemple suivant si la route de l'application /hello-servlet est sollicitée avec le verbe HTTP GET, alors la méthode doGet est invoquée avec deux paramètres, la requête et la réponse.

```
package iut.nantes.demo;

import java.io.*;

import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;

@WebServlet(name = "helloServlet", value = "/hello-servlet")
public class HelloServlet extends HttpServlet {
    private String message;

    public void init() {
        message = "Hello World!";
    }

    public void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException {
        response.setContentType("text/html");

        // Hello
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h1>" + message + "</h1>");
        out.println("</body></html>");
    }

    public void destroy() {
    }
}
```

2.2.2.2. Un JSP

Un JSP est une technique qui permet à partir d'un langage XML de créer des servlets. Ici nous aurons un titre et un lien vers la servlet précédente.

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html>
<head>
    <title>JSP - Hello World</title>
</head>
<body>
<h1><%= "Hello World!" %>
</h1>
<br/>
<a href="hello-servlet">Hello Servlet</a>
</body>
</html>
```

2.2.3. La compilation et le déploiement

Nous avons du code, il nous faut le compiler pour obtenir du byte code, des .class et déployer ce byte code sur le serveur.

2.2.3.1. Le moteur de production

Il nous faut un moyen d'automatiser la production du code exécutable (compilation et assemblage), avec une gestion et une détection des dépendances, nous allons utiliser gradle mais vous trouverez également des solutions avec maven ou encore ant.

Structure d'un projet gradle :

```
gradle-project
### app
#    ### build.gradle
#    ### ...
```

```
### settings.gradle
### gradle
# ### ...
### build
### gradlew
### gradlew.bat
```

gradlew

un script qui télécharge et exécute automatiquement la version de Gradle appropriée pour un projet.

build

la configuration au niveau du projet

setting.gradle

pour intégrer le projet et construire une identité

gradle

Exécutables (wrapper) » et version

Le fichier qui nous intéresse est build.gradle, nous y trouvons à minima :

l'identifiant du groupe et la version

```
group 'iut.nantes'
version '1.0-SNAPSHOT'
```

des plugins

```
plugins {
    id 'war'
}
```

Nous aurons besoin de war pour le déploiement

de dépôts

```
repositories {
    mavenCentral()
}
```

de dépendances

```
dependencies {
    compileOnly('jakarta.servlet:jakarta.servlet-api:6.2.0-M2')
}
```

des versions

```
java {
    sourceCompatibility = JavaVersion.VERSION_21
    targetCompatibility = JavaVersion.VERSION_21
}
```

Gradle repose sur la notion de tâche, une action atomique pour une compilation. Vous pouvez obtenir la liste des tâches avec **./gradlew tasks**.

2.2.3.2. Les fichiers de déploiement

Pour déployer du code sur un serveur tomcat, il faut déposer le code compilé, avec les descripteurs, les éventuelles ressources et bibliothèques dans un dossier ou un sous dossier du répertoire webapps du serveur.

¹Cette version est celle installée dans les salles, nous pouvons utiliser une version plus récente en passant par notre IDE.

Le serveur tomcat dont nous disposons est configuré pour déployer sans redémarrage à chaque dépôt dans /usr/local/tomcat/webapps que nous avons monté dans répertoire local ./tomcat/webapps.

A vous trouver la tâche gradle pour générer le war, puis de déposer le war créé dans le répertoire webapps de tomcat, le déploiement devrait être automatique. Vous, pouvez tester avec l'URL du la page JSP, le lien conduisant à la servlet : `http://localhost:8080/demo-1.0-SNAPSHOT/index.jsp` *Le localhost n'étant pas toujours disponible à l'IUT vous devrez le remplacer par votre adresse IP 172.16.XX.XX obtenue avec la commande **ip a**.*

demo-1.0-SNAPSHOT est la concaténation de rootProject.name de settings.gradle et de version de build.gradle, il est possible de choisir le nom dans build.gradle en utilisant la propriété archiveFileName de war, à vous de tester :

```
war {  
    archiveFileName = 'demo.war'  
}
```

Si vous modifiez votre code, il vous faut le redéployer en supprimant le war déposé, puis en le remplaçant, nous pouvons automatiser tout cela avec des tâches.

```
tasks.register('deleteMyWar', Delete) {  
  
    doLast {  
        println("Suppression du war : ./tomcat/webapps/${war.archiveFileName.get()}")  
        println("Done")  
    }  
    delete files("./tomcat/webapps/${war.archiveFileName.get()}")  
}
```

```
tasks.register('copyMyWar', Copy){  
  
    doLast {  
        println("Copy ${war.archiveFileName.get()} from ${project.buildDir}/libs to ./tomcat/webapps")  
        println("Done")  
    }  
    from "${project.buildDir}/libs"  
    into  "./tomcat/webapps"  
    include "${war.archiveFileName.get()}"  
}
```

Il ne reste plus qu'à supprimer, le war, le recréer et le copier en faisant attention que la suppression soit finie.

Chapitre 2. JEE (servlet, JSP, JPA)

Nous allons commencer par un tutori

1. Apprentissage de java

1.1. Différences entre java et Kotlin

Java est plus ancien que Kotlin avec une syntaxe plus verbeuse mais un meilleur support. Java est comme Kotlin un langage objet, fortement typé et exécuté sur une JVM. Voici avant quelques différences :

null safety

Java autorise l'affectation de valeurs nulles aux objets. Par conséquent, l'accès à un membre d'une référence nulle provoque à l'exécution une exception : `NullPointerException`.

Extension Functions

Les fonctions d'extension n'existent pas en java, le seul moyen pour étendre les méthodes est d'hériter.

Typage strict vs. typage statique

Java est un langage à typage strict, ce qui signifie que toutes les variables sont déclarées avec un type lors de leur création. En revanche, Kotlin est un langage à typage statique qui peut déterminer le type d'une variable en fonction du type de la valeur qui lui est assignée (inférence de type).

Casting

En Kotlin le compilateur peut générer automatiquement les transtypages, en java ils devront être explicites.

Programmation fonctionnelle

Kotlin est à la fois orienté objet et fonctionnel, ce qui n'était pas le cas de java pour les premières versions de java. Les lambda existent pas les fonctions d'extensions et les fonctions déléguées.

Coroutines

Java dispose des threads mais pas des coroutines

Classes scellées

En java, les classes sont ouvertes par défaut, si vous souhaitez limiter l'héritage, il faudra utiliser les mots clefs `final` ou `sealed`.

1.2. Un premier code en java

Vous pouvez utiliser l'IDE de votre choix et créer un projet java classique, nous n'aurons pas besoin de JEE pour le moment. Un projet gradle avec des tests est fourni.

1.2.1. Un tutoriel

Je vous propose de commencer par un tutoriel officiel java portant sur les classes et les objets : <https://docs.oracle.com/javase/tutorial/java/java00/index.html>. Vous pouvez parcourir rapidement la partie cours mais n'oubliez pas la partie exercices. Vous devrez aller jusqu'aux énumérations. Par rapport à Kotlin, vous devriez apprendre qu'une classe est ouverte par défaut, que l'héritage s'exprime avec le mot clef `extends`, que la

réalisation d'une interface est exprimée avec `implements` que le chaînage des constructeurs s'exprime avec `super()`, que les constructeurs peuvent-être surchargés et appelé via `this()` que les objets sont instanciés avec `new`. Les "compagnions" peuvent être assimilés aux attributs statique et aux classes statiques imbriquées mais avec sans accès directe, il faut passer par le nom de la classe.

1.2.2. La promotion : Classe et héritage

Nous allons coder une Promotion composée d'étudiants, à chaque étape, vous devrez passer les tests.

1.2.2.1. Un étudiant

Créer une classe `Etudiant` avec deux attributs privés `no` une string et une moyenne un flottant, les méthode `getNo()` et `getMoyenne()` permettront de consulter les attributs, un constructeur permettra de construire un étudiant avec un `no` et une moyenne choisie, si la moyenne n'est pas fournie, elle sera supposée égale à 10.

Figure 2.1. Classe Etudiant

Etudiant		
Ⓜ	Etudiant(String, Float)	
Ⓜ	Etudiant(String)	
f	moyenne	Float
f	no	String
Ⓜ	getNo()	String
Ⓜ	toString()	String
Ⓜ	getMoyenne()	Float

Créer un classe avec un main pour créer deux étudiants en utilisant les deux constructeurs et afficher les propriétés.

1.2.2.2. Une énumération

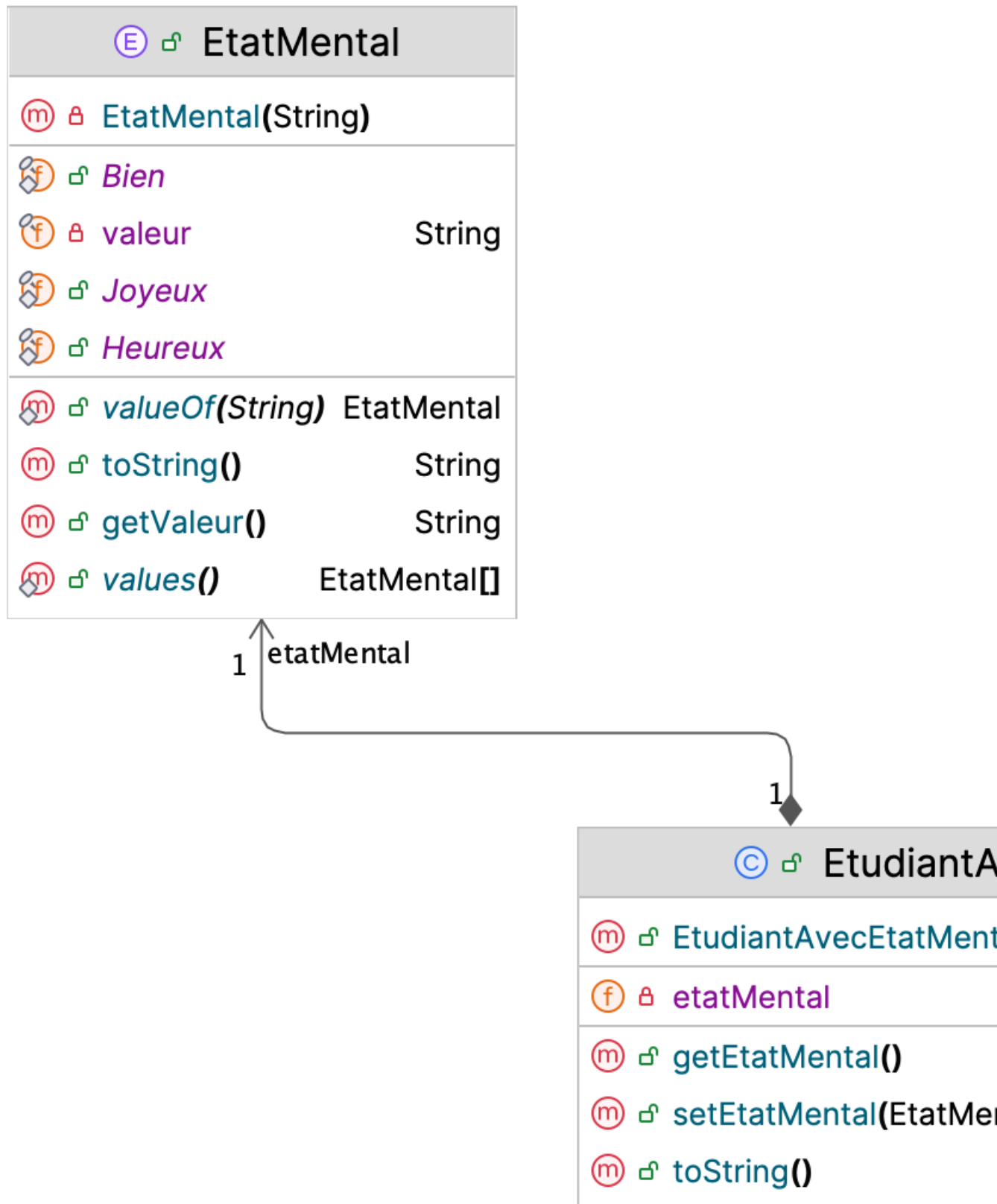
Vous trouvez ici la document java d'une énumération : <https://docs.oracle.com/javase/tutorial/java/java00/enum.html>. Cette énumération dispose d'un constructeur et d'un attribut valeur de type string. Les méthode `values()` et `valueOf()` sont natives et ne sont pas à coder.

Figure 2.2. EtatMental

  EtatMental		
 	EtatMental(String)	
 	<i>Bien</i>	
 	<i>Joyeux</i>	
 	<i>Heureux</i>	
 	<i>valeur</i>	String
 	toString()	String
 	getValeur()	String
 	valueOf(String)	EtatMental
 	values()	EtatMental[]

1.2.2.3. Un étudiant avec un état mental

Créer une classe fille EtatMental qui représente un Etudiant avec un EtatMental.

Figure 2.3. EtudiantAvecEtatMental

1.2.2.4. Connaître le nombre de tous les étudiants créés

En utilisant un attribut et une méthode static pour comptabiliser et récupérer les étudiants créés.

Figure 2.4. Etudiant avec comptage

© Etudiant	
Ⓜ Etudiant(String)	
Ⓜ Etudiant(String, Float)	
Ⓣ moyenne	Float
Ⓣ nb	Integer
Ⓣ no	String
Ⓜ getNo()	String
Ⓜ nombreEtudiantCrees()	Integer
Ⓜ toString()	String
Ⓜ getMoyenne()	Float

1.2.2.5. La promotion

Implémenter et tester une promotion d'étudiants, un étudiant caractérisé par son numéro ne pourra être ajouté deux fois cela devra déclencher une Exception. La suppression d'un étudiant non membre renverra quand à elle false. Vous devrez également fournir la moyenne de la promotion, si la promotion est vide alors la moyenne sera nulle.

1.2.3. Mise en oeuvre du design pattern DAO

Nous allons mettre en oeuvre le design pattern DAO, de manière classique pour en utilisant les Optionals, une solution proposée par java pour répondre au problème de sûreté des nulls.

1.2.3.1. Approche classique

Le travail est à réaliser dans le package old, l'interface DAO, la classe Product ainsi que les jeux de tests sont fournis, vous devez coder la classe ProductMemoryDao qui implémente un DAO pour des products. Je vous conseil d'utiliser pour son implémentation une HashMap (<https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html>).

1.2.3.2. Approche Optional

Les "Optional" ont été créés en java 8 pour répondre au problème des nulls pointeur exception (<https://www.oracle.com/technical-resources/articles/java/java8-optional.html>).

Les "Optional" encapsule un type pour forcer la gestion des nulls. Il constitue avec les stream (<https://docs.oracle.com/javase/8/docs/api/java/util/stream/Stream.html>) un moyen d'avoir une approche fonctionnelle sans null pointeur exception. Il est à noter que les "Optional" ne sont pas sérialisables.

Voici un mini guide de survie :

Une optional vide

```
Optional<String> myOptional = Optional.empty();
```

Une option sur un élément non null

```
Optional<String> myOptional = Optional.of(element);
```

Une optional sur un élément potentiellement null

```
Optional<String> myOptional = Optional.ofNullable(element);
```

Cette méthode permet également de créer une optional qui encapsule null

```
Optional<String> myOptional = Optional.ofNullable(null);
```

Présence d'un élément

```
myOptional.isEmpty()
```

ou

```
myOptional.isPresent()
```

Récupérer un élément avec possibilité de NoSuchElementException

```
myOptional.get()
```

Récupérer un élément avec une valeur par défaut

```
myOptional.orElse(ValeurParDefaut)
```

Récupération avec une valeur par défaut issue d'un Supplier

Un supplier est souvent utilisé sous sa forme lambda (<https://docs.oracle.com/javase/8/docs/api/java/util/function/Supplier.html>).

Cette approche permet de renvoyer une valeur par défaut issue d'un supplier si l'optional est vide.

```
myOptional.orElseGet(() -> ValeurParDefaut))
```

Cette approche est plus performante que la précédente en ce sens que la lambda n'est exécutée que si l'optional n'est pas vide.

Lever une exception la valeur est absente

```
Optional.ofNullable(element).orElseThrow();
```

ou

```
Optional.ofNullable(element)
    .orElseThrow(() -> new Exception(""));
```

Les filtres

```
myOptional.filter(lambda)
```

Les filtres répondent au design pattern builder

```
myOptional.filter(lambda).filter(lambda).isPresent()
```

Les mappings

```
myOptional.map(Methode).orElse(Valeur)
```

Implémenter dans le package Optional l'interface DAO qui a été modifiée avec :

```
Optional<T> get(long id);
```

Créer également des jeux de tests.

Cette nouvelle est plus adaptée à l'utilisation de la Java Persistence API que nous utiliserons plus tard et dans laquelle la méthode `find` renvoie des `Optionals`.

2. Les servlet

2.1. Cycle de vie

Une servlet sera pour nous une classe fille de `HttpServlet` qui s'exécute sur un serveur web pour recevoir des requêtes HTTP (Get, Post) et fournir une réponse HTML.

Pour exécuter un servlet, il faut utiliser une requête HTTP, si le servlet est associé à l'URL alors si c'est la première sollicitation, le serveur l'instancie. La servlet reste en mémoire jusqu'à l'arrêt du serveur. La servlet peut être appelée par plusieurs threads lancés par le serveur.

Le cycle de vie d'une servlet Java est géré par le conteneur web (comme Tomcat). Il comprend trois phases principales : l'initialisation via `init()`, le traitement des requêtes via `service()` (qui aiguille vers `doGet()` ou `doPost()`), et la destruction via `destroy()`, de sa création à sa suppression.

Créer un package `cycle` et créer dans ce package une servlet nommée `CycleServlet` associé au chemin d'URL / `cycle-servlet`, dans cette servlet redéfinir le constructeur, les méthodes `init()`, `init(ServletConfig config)`, `doGet`, `doPost`, `doPut`, `doPatch`, `doDelete`, `destroy` dans chaque méthode. Déployer la servlet et la tester, vous pouvez utiliser les commandes `curl` :

```
curl --noproxy "*" -X GET "http://localhost:8888/demo/cycle-servlet
curl --noproxy "*" -X POST "http://localhost:8888/demo/cycle-servlet
curl --noproxy "*" -X PUT "http://localhost:8888/demo/cycle-servlet
curl --noproxy "*" -X PATCH "http://localhost:8888/demo/cycle-servlet
curl --noproxy "*" -X DELETE "http://localhost:8888/demo/cycle-servlet
```

Pour accéder au log de tomcat, vous pouvez utiliser `terminal.sh` et accède au log avec **`tail -f logs/nom_du_log`**. Pour accéder aux logs du terminal, il vous faudra utiliser **`podman logs tomcat_container`**.

Si vous utilisez `super.doUnVerbeHTTP()`, comme `super.doGet()`, il renvoie par défaut un code 405, vous pouvez l'ignorer. De même, vous ne pourrez observer le `destroy()`.

2.2. Recevoir des paramètres

//TODO

3. JSP

4. JPA

Chapitre 3. Une première application MVC

Nous sommes suffisamment robuste pour commencer la programmation coté serveur.

Chapitre 4. Utilisation du framework MVC spring boot